Introduction

In many real-world applications, tasks are performed sequentially, where one task must complete before the next one begins. While this approach is simple, it may not always be the most efficient, especially when tasks take time to complete. In this article, we will explore how multithreading can help optimize such scenarios by allowing multiple tasks to run concurrently, improving efficiency.

Let's explore a simple problem where multiple operations need to be executed sequentially, each taking a certain amount of time to complete.

Problem Statement

Imagine you've placed an order, and the next steps involve sending three notifications:

- **Send an SMS:** This takes 2 seconds.
- **Send an Email:** This takes 3 seconds.
- **Send ETA (Estimated Time of Arrival):** This takes 5 seconds.

To simulate these delays in Java, we will use the `Thread.sleep()` from the `java.lang`.

Let's look at the code that tries to solve this problem performing the tasks in a sequential manner (without using **Multithreading**).

Code Simulating Delays Sequentially Without Multithreading

import java.util.*;

```
// OrderService class

class OrderService {


   // Main method simulates placing an order and executing tasks sequentially

   public static void main(String[] args) throws InterruptedException {

      System.out.println("Placing order...\n");


      // Send SMS and simulate the delay of 2 seconds

      sendSMS();

      System.out.println("Task 1 done.\n");


      // Send Email and simulate the delay of 3 seconds

      sendEmail();

      System.out.println("Task 2 done.\n");


      // Calculate ETA (Estimated Time of Arrival) with a delay of 5 seconds

      String eta = calculateETA();

      System.out.println("Order placed. Estimated Time of Arrival: " + eta);

      System.out.println("Task 3 done.\n");

   }
```

```
// Method to simulate sending SMS with a 2-second delay
private static void sendSMS() {
    try{
        Thread.sleep(2000); // Delay of 2 seconds
        System.out.println("SMS Sent!");
    }
    catch (InterruptedException e) {
        e.printStackTrace();
    }
}

// Method to simulate sending Email with a 3-second delay
private static void sendEmail(){
    try{
        Thread.sleep(3000); // Delay of 3 seconds
        System.out.println("Email Sent!");
    }
    catch (InterruptedException e) {
        e.printStackTrace();
    }
```

```
    }

    // Method to simulate calculating the ETA with a 5-second delay
    private static String calculateETA() {
        try {
            Thread.sleep(5000); // Delay of 5 seconds
        }
        catch (InterruptedException e) {
            e.printStackTrace();
        }

        return "25 minutes"; // Returning the calculated ETA
    }
}

// Main class
class Main {

    public static void main(String[] args) {
        // Initiating the order processing by calling the OrderService main
method
        try {
```

```
            OrderService.main(args); // Call the OrderService's main method
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

Understanding the Issue

In real-world applications, executing tasks sequentially like this can lead to significant inefficiencies, especially when tasks involve **waiting periods**, such as sending SMS, emails, or calculating ETAs. In our current approach, each operation must complete before the next one begins, causing **unnecessary delays** and making the system slower than it needs to be.

For instance, while waiting for an SMS to be sent, the program is effectively **"idle"**, unable to do anything else. This results in wasted time, especially in systems where responsiveness and speed are critical. By running tasks sequentially, we also limit the system's ability to handle multiple operations **concurrently**, which can lead to poor user experiences, slower processing times, and bottlenecks in performance.

This is where **multithreading** comes into play. By executing tasks concurrently, we can ensure that while one task is waiting (like sending an SMS), others can proceed without delay. Instead of idly waiting for each operation to finish, multithreading allows the system to be more responsive, efficiently utilizing time and resources. In the next section, we will explore how multithreading can help overcome these issues, making the system more efficient and responsive.

Implementing Multithreading by Extending the `Thread` Class

In Java, multithreading can be implemented using many different ways. One such way is by creating a **subclass** of the **Thread** class and overriding the **run()** method. Each thread will execute the **run()** method, allowing concurrent execution of tasks.

Key Concepts

- **`Thread` Class:** The `Thread` class is used to represent a thread in Java. By extending the `Thread` class, we can override its **run()** method to specify the operations or actions that the thread will perform. In other words, the **run()** method contains the code that defines what the thread will do when it is executed.
- **start() method:** This method is used to start a new thread of execution. It internally calls the **run()** method.
- **join() method:** This method makes the main thread wait for the thread to finish before proceeding.

Code

```
import java.util.*;


// Creating a subclass of Thread to send SMS
class SMSThread extends Thread {
    public void run() {
        try {
            Thread.sleep(2000); // 2-second delay for SMS
```

```
            System.out.println("SMS Sent using Thread.");
        } catch(InterruptedException e) {
            e.printStackTrace();
        }
    }
}

// Creating a subclass of Thread to send Email
class EmailThread extends Thread {
    public void run() {
        try {
            Thread.sleep(3000); // 3-second delay for Email
            System.out.println("Email Sent using Thread.");
        } catch(InterruptedException e) {
            e.printStackTrace();
        }
    }
}

// Creating a subclass of Thread to calculate ETA
class ETACalculationThread extends Thread {
```

```
    public void run() {
        try {
            Thread.sleep(5000); // 5-second delay for ETA calculation
            System.out.println("ETA Calculated using Thread. Estimated Time:
25 minutes.");
        } catch(InterruptedException e) {
            e.printStackTrace();
        }
    }
}

class Main {
    public static void main(String[] args) {
        // Create thread objects for SMS, Email, and ETA Calculation
        SMSThread smsThread = new SMSThread();
        EmailThread emailThread = new EmailThread();
        ETACalculationThread etaThread = new ETACalculationThread();

        System.out.println("Task Started.\n");

        // Start all threads
        smsThread.start();
```

```
        System.out.println("Task 1 ongoing...");


        emailThread.start();
        System.out.println("Task 2 ongoing...");


        etaThread.start();
        System.out.println("Task 3 ongoing...");


        // Wait for all threads to finish
        try {
            smsThread.join();
            emailThread.join();
            etaThread.join();
            System.out.println("All tasks completed.");
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

Explanation

- The `SMSThread`, `EmailThread` and `ETACalculationThread` classes extend `Thread` and override the **run()** method to simulate sending SMS and Email with a delay.
- The **start()** method initiates the execution of each thread.
- **join()** ensures the main thread waits for both SMS and Email tasks to finish before continuing.

The above code demonstrates three concurrent tasks: sending an SMS, sending an email, and calculating the ETA, each using a separate thread by extending the `Thread` class.

Demerit: No Return Type in the `run()` Method

A limitation of the `Thread` class in Java is that the `run()` method cannot return a value (because it's void). This means that if you need to return a result, like an ETA string, you cannot directly do so from the **run()** method.

This is a key drawback of using the `Thread` class for tasks that require returning values.

Note: The methods to get a result from a thread is discussed in the later part of the article.

Implementing Multithreading by Using the `Runnable` Interface

In Java, another way to implement multithreading is by using the **`Runnable` interface**. Unlike the `Thread` class approach, where we extend `Thread`, the `Runnable` interface allows us to define a task that can be executed by multiple threads. The `Runnable` interface represents a task that can be executed concurrently, but it does not manage the thread itself, we need to pass it to a `Thread` object for execution.

Key Concepts

- **`Runnable` Interface:** The `Runnable` interface is a functional interface that contains a single method, **run()**. By implementing this interface, we can define the task to be executed by a thread. The **run()** method contains the code that will be executed concurrently.
- **run() Method:** Unlike the `Thread` class, where we override **run()** directly, in this case, we provide the task's code by implementing the **run()** method of the `Runnable` interface.
- **`Thread` Class:** While `Runnable` defines the task, the actual thread of execution is created by passing the `Runnable` object to a `Thread` object. The `Thread` class is responsible for managing and executing the thread.
- **start() Method:** The `start()` method is used to begin the thread's execution. It internally calls the `run()` method.
- **join() Method:** Similar to the `Thread` class, `join()` is used to ensure that the main thread waits for the other threads to finish before continuing.

Code

```
import java.util.*;



// Implementing the Runnable interface for sending SMS
```

```
class SMSTask implements Runnable {
    public void run() {
        try {
            Thread.sleep(2000); // 2-second delay for SMS
            System.out.println("SMS Sent using Runnable.");
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

// Implementing the Runnable interface for sending Email
class EmailTask implements Runnable {
    public void run() {
        try {
            Thread.sleep(3000); // 3-second delay for Email
            System.out.println("Email Sent using Runnable.");
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
```

```
}

// Implementing the Runnable interface for calculating ETA
class ETATask implements Runnable {
    public void run() {
        try {
            Thread.sleep(5000); // 5-second delay for ETA calculation
            System.out.println("ETA Calculated using Runnable. Estimated Time: 25 minutes.");
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

class Main {
    public static void main(String[] args) {
        // Create Runnable objects for SMS, Email, and ETA calculation
        SMSTask smsTask = new SMSTask();
        EmailTask emailTask = new EmailTask();
        ETATask etaTask = new ETATask();
```

```
// Create Thread objects and pass the Runnable tasks to them
Thread smsThread = new Thread(smsTask);
Thread emailThread = new Thread(emailTask);
Thread etaThread = new Thread(etaTask);

System.out.println("Task Started.\n");

// Start all threads
smsThread.start();
System.out.println("Task 1 ongoing...");

emailThread.start();
System.out.println("Task 2 ongoing...");

etaThread.start();
System.out.println("Task 3 ongoing...");

// Wait for all threads to finish
try {
    smsThread.join();
    emailThread.join();
```

```
            etaThread.join();

            System.out.println("All tasks completed.");

        } catch (InterruptedException e) {

            e.printStackTrace();

        }

    }

}
```

Explanation

- `Runnable` **Interface:** Each task (SMS, Email, ETA calculation) is represented by a class implementing the `Runnable` interface. The **run()** method defines the task's behavior.
- **Thread Creation:** Instead of subclassing `Thread`, we create instances of `Thread` and pass a `Runnable` object to the constructor. The `Thread` class is responsible for managing and executing the task defined in the **run()** method.
- **Thread Execution:** The **start()** method begins the execution of the thread, invoking the **run()** method in a separate thread of execution. The **join()** method ensures the main thread waits for all tasks to finish before printing "**All tasks completed.**"

The above code demonstrates three concurrent tasks: sending an SMS, sending an email, and calculating the ETA, each using a separate thread by implementing the `Runnable` interface.

Advantages of Using `Runnable`

- **Separation of Concerns:** By using `Runnable`, we separate the task logic from the thread management. This is particularly useful when you need to share the same task between multiple threads.
- **Flexibility:** Since `Runnable` can be passed to any `Thread` object, it provides more flexibility compared to extending the `Thread` class. You can implement multiple interfaces along with `Runnable`, whereas extending Thread limits you to just the `Thread` class.

Demerit: No Return Type in the `run()` Method

A limitation of the `Runnable` interface in Java is that the **run()** method cannot return a value (because it's **void**). This means that if you need to return a result, like an ETA string, you cannot directly do so from the `run()` method.

This is a key drawback of using the `Runnable` interface for tasks that require returning values.

Fire and Forget: A Pattern in Multithreading

Both the `Runnable` interface and the `Thread` class, as discussed, naturally follow the **Fire and Forget** pattern.

In this pattern, tasks are initiated (fired), but the system doesn't wait for a result or confirmation of their completion. Instead, the tasks are executed independently in the background, and the caller doesn't need to know when or how they finish.

This approach works well when the result of the task is not critical or when tasks do not need to communicate their results back to the main thread. For instance, sending SMS, emails, or calculating an ETA can be handled using the `Runnable` method without waiting for confirmation of completion.

However, while the `Runnable` method is great for many cases, it does have limitations, especially when the result of the task is needed. This is where the `Callable` interface comes into play. In the next section, we will explore how to use `Callable` to handle tasks that require returning results.

Implementing Multithreading by Using the `Callable` Interface and `Future`

In Java, the `Callable` interface provides an enhanced way to implement multithreading when you need tasks to return a result. Unlike the previously discussed methods, which does not return any result, `Callable` allows the thread to return a value once the task completes. However, since the **run()** method of `Runnable` cannot return a result (it's void), the `Callable` interface provides a **call()** method, which can return any type of value.

In addition, `Future` is an interface that represents the **result** of an

asynchronous computation. It provides methods to check the status of a task and retrieve the result once it's available, thus allowing you to avoid the limitations of the `Thread` and `Runnable` approaches.

Key Concepts

- `Callable` **Interface:** The `Callable` interface is similar to `Runnable`, but its **call()** method is designed to return a result, unlike **run().** It also allows for throwing exceptions, which is useful for more complex tasks.
- **call() Method:** The **call()** method in the `Callable` interface contains the code for the task and can return any value (unlike **run()** in `Runnable`). It also allows the task to throw checked exceptions.
- `ExecutorService`**:** While you can directly use `Thread` with `Callable`, it's recommended to use `ExecutorService` for better thread management. The `ExecutorService` provides a method called **submit()** to submit a `Callable` task and returns a `Future` object that can be used to retrieve the result later.

Code

```
import java.util.concurrent.*;


// Implementing Callable to calculate ETA (only this task requires a return value)
class ETACalculationTask implements Callable<String> {
    public String call() throws InterruptedException {
        Thread.sleep(5000); // Simulate 5-second delay for ETA calculation
        System.out.println("Calculation ETA using Callable.");
        return "ETA: 25 minutes"; // Return the ETA result
    }
```

```
}

// Implementing Runnable to send SMS (no return value required)
class SMSTask implements Runnable {
    public void run() {
        try {
            Thread.sleep(2000); // Simulate 2-second delay for SMS
            System.out.println("SMS Sent using Runnable.");
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

// Implementing Runnable to send Email (no return value required)
class EmailTask implements Runnable {
    public void run() {
        try {
            Thread.sleep(3000); // Simulate 3-second delay for Email
            System.out.println("Email Sent using Runnable.");
        } catch (InterruptedException e) {
```

```
            e.printStackTrace();
        }
    }
}

class Main {
    public static void main(String[] args) {
        // Create ExecutorService to manage threads
        ExecutorService executorService = Executors.newFixedThreadPool(3);

        // Create Callable task for ETA calculation and Runnable tasks for SMS and Email
        SMSTask smsTask = new SMSTask();
        EmailTask emailTask = new EmailTask();
        ETACalculationTask etaTask = new ETACalculationTask();

        // Submit tasks and get Future objects for the ETA task
        Future<String> etaResult = executorService.submit(etaTask);

        // Submit the SMS and Email tasks (no result required)
        executorService.submit(smsTask);
        executorService.submit(emailTask);
```

```
        try {
            // Get the result from the Future object for ETA
            System.out.println(etaResult.get()); // Wait for ETA task to finish and print result
        } catch (InterruptedException | ExecutionException e) {
            e.printStackTrace();
        }

        // Shutdown the ExecutorService
        executorService.shutdown();
    }
}
```

Explanation

- `Callable` **Interface:** Each task (SMS, Email, and ETA calculation) implements the `Callable` interface, and the **call()** method contains the task's code.
- **Thread Management with** `ExecutorService`**:** We use `ExecutorService` to submit the tasks. This manages the threads efficiently and allows us to easily handle the results via Future.

- **Result Retrieval with `Future`:** The **submit()** method of ExecutorService returns a `Future` object, which we use to get the result of the task using the **get()** method.
- **shutdown():** Finally, we call **shutdown()** on `ExecutorService` to gracefully terminate the thread pool after all tasks are completed.

Using `Future` to Retrieve Results

When you submit a task using `Callable`, the `ExecutorService` returns a `Future` object. The `Future` interface provides methods to check the status of a task and retrieve its result once it completes.

Key Concepts

`Future` **Interface**: The `Future` interface represents the result of an asynchronous computation. You can use it to:

- **Retrieve the result:** The **get()** method blocks the main thread until the task completes and returns the result.
- **Check the status:** Methods like **isDone()** and **isCancelled()** help you check if the task is completed or canceled.
- **Cancel the task:** The **cancel()** method attempts to cancel the task before it finishes.

As shown in the code above, the **result** after the ETA Calculation is retrieved using `Future` interface. Another method to retrieve results from `Callable()` is to use `FutureTask`.

Using `FutureTask` to Retrieve Results

`FutureTask` is a concrete implementation of the `Future` interface. It is used to wrap a `Callable` task and execute it asynchronously. The benefit of using `FutureTask` is that it can be executed like a `Runnable` (i.e., passed to a `Thread`), and it can also store and return the result of the task.

Key Concepts

- `FutureTask` **Class:** It implements both `Runnable` and `Future`, making it a versatile tool for concurrent tasks. You can submit a `FutureTask` to a thread pool or execute it directly via a `Thread`.
- **get() Method:** Just like `Future`, `FutureTask` allows you to retrieve the result of the task via **get()**.

Example Code: Using FutureTask for Retrieval

```
import java.util.*;


// Implementing Callable to calculate ETA (only this task requires a return value)
class ETACalculationTask implements Callable<String> {
    public String call() throws InterruptedException {
        Thread.sleep(5000); // Simulate 5-second delay for ETA calculation
        System.out.println("ETA calculated using Callable.");
        return "ETA: 25 minutes"; // Return the ETA result
    }
}
```

```
// Implementing Runnable to send SMS (no return value required)
class SMSTask implements Runnable {
    public void run() {
        try {
            Thread.sleep(2000); // Simulate 2-second delay for SMS
            System.out.println("SMS Sent using Runnable.");
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

// Implementing Runnable to send Email (no return value required)
class EmailTask implements Runnable {
    public void run() {
        try {
            Thread.sleep(3000); // Simulate 3-second delay for Email
            System.out.println("Email Sent using Runnable.");
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
```

```
    }
}

class Main {
    public static void main(String[] args) {
        // Create FutureTask object for ETA calculation task (since it returns a result)
        FutureTask<String> etaTask = new FutureTask<>(new ETACalculationTask());

        // Create Runnable tasks for SMS and Email
        SMSTask smsTask = new SMSTask();
        EmailTask emailTask = new EmailTask();

        // Create Thread objects to run all tasks
        Thread etaThread = new Thread(etaTask);
        Thread smsThread = new Thread(smsTask);
        Thread emailThread = new Thread(emailTask);

        System.out.println("Task Started.\n");

        // Start all threads
```

```
        etaThread.start();
        System.out.println("Task 1 ongoing...");

        smsThread.start();
        System.out.println("Task 2 ongoing...");

        emailThread.start();
        System.out.println("Task 3 ongoing...");

        try {
            // Get the result from the FutureTask for ETA
            System.out.println((String)etaTask.get()); // Wait for ETA task to
finish and print result

            // Wait for SMS and Email tasks to finish (no result needed)
            smsThread.join();
            emailThread.join();
        } catch (InterruptedException | ExecutionException e) {
            e.printStackTrace();
        }

        System.out.println("All tasks completed.");
```

```
    }
}
```

Other Ways to Implement Multithreading in Java

In addition to the methods we've discussed so far, there are other ways to implement multithreading in Java. These methods offer different levels of simplicity and flexibility, depending on the task at hand.

1. Directly Defining a `Runnable` Instance

In Java, you don't always need to create a separate class to implement the `Runnable` interface. You can define a `Runnable` instance directly using an anonymous class or a simple lambda expression (discussed next). This is a concise and efficient way to handle simple tasks without needing an entire class structure.

Here's how you can define a `Runnable` directly:

```
class Main {
    public static void main(String[] args) {
        // Define Runnable directly as an anonymous class
        Runnable task = new Runnable() {
            @Override
            public void run() {
                try {
```

```
            Thread.sleep(2000); // Simulate delay
            System.out.println("Task completed using direct Runnable.");
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
};

// Create and start the thread
Thread thread = new Thread(task);
thread.start();
  }
}
```

In this example:

- The `Runnable` interface is implemented directly within the **main** method using an anonymous class.
- The **run()** method defines the task to be executed in the new thread.

This approach is particularly useful for short-lived tasks where you don't need to create a separate class.

---

2. Using Lambda Expressions (Java 8 and Above)

With **Java 8**, the introduction of **lambda expressions** made it easier to write concise, functional-style code. Lambda expressions provide a cleaner and more readable way to define `Runnable` tasks without the boilerplate code of an anonymous class.

Here's how you can use a lambda expression to define a `Runnable`:

```
class Main {
    public static void main(String[] args) {
        // Define Runnable using a lambda expression
        Runnable task = () -> {
            try {
                Thread.sleep(2000); // Simulate delay
                System.out.println("Task completed using Lambda expression.");
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        };

        // Create and start the thread
        Thread thread = new Thread(task);
```

```
        thread.start();
    }
}
```

In this example:

- The `Runnable` interface is implemented using a lambda expression, making the code shorter and easier to read.
- The task (defined in the **run()** method) is executed in a new thread when **start()** is called.

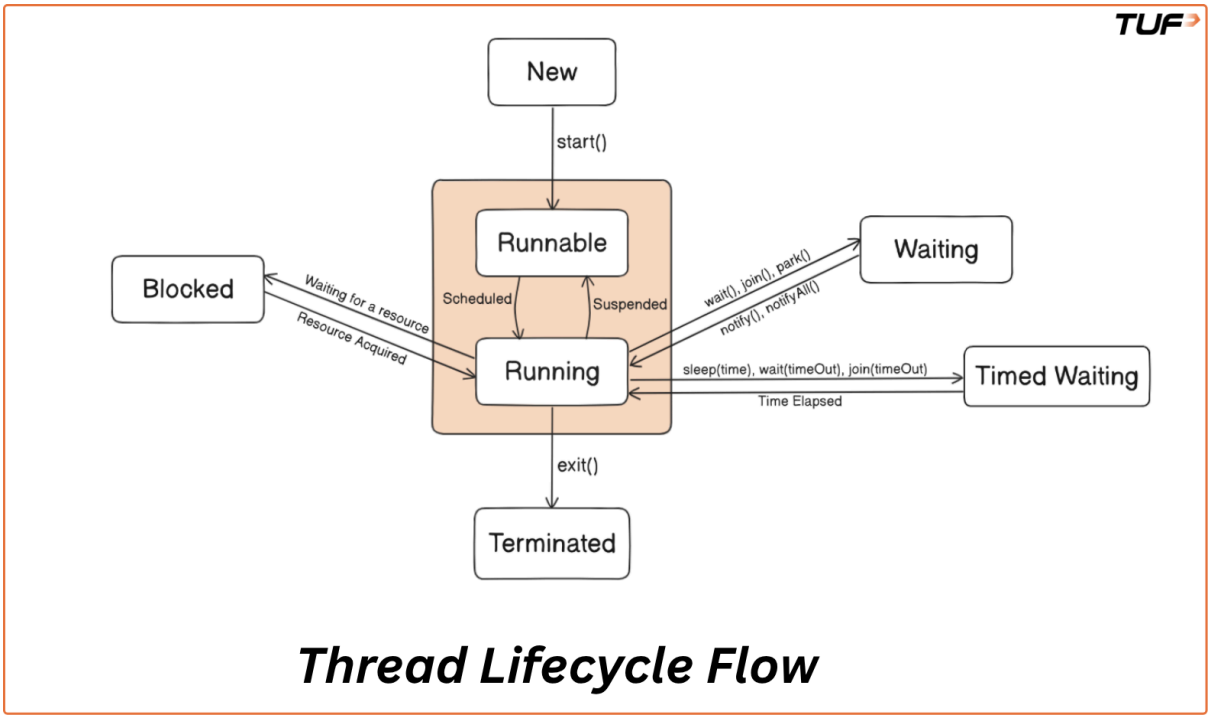
Advantages of Lambda Expressions

- **Conciseness:** Lambda expressions reduce the need for boilerplate code (e.g., no need for `new Runnable()` and method overrides).
- **Readability:** The code is more readable and expresses the task logic directly.

Lambda expressions are especially helpful when you need to pass short tasks to a thread and when you want to minimize the verbosity of the code.

Thread Lifecycle Flow

The lifecycle of a thread in Java follows a specific flow as shown in the diagram:



States in Thread Lifecycle

A thread undergoes various states during its execution. These states represent different phases a thread can be in from its creation to termination. Below are the key states in the lifecycle:

- **New:** The thread is created but not started yet.
- **Runnable:** The thread is eligible to run but is not necessarily executing. It transitions to **Running** when the CPU picks it for execution.
- **Running:** The thread is actively executing.
- **Blocked:** The thread is waiting for a resource or lock.
- **Waiting:** The thread is waiting indefinitely for another thread to perform an action (e.g., **wait()** or **join()**).
- **Timed Waiting:** The thread is waiting for a specific amount of time (e.g., **sleep()** or **join(time)**).
- **Terminated: The thread has completed its execution or is stopped.**The thread has completed its execution or is stopped.

State Transitions

- **New → Runnable: When start() is called.**
- **Runnable → Running:** When the thread is scheduled to execute.
- **Running → Blocked:** When the thread waits for a resource.
- **Blocked → Runnable:** When the resource becomes available.
- **Running → Waiting:** When the thread calls **wait()**, **join()**, or **lock()**.
- **Waiting → Runnable:** When the thread is notified (**notify()**, **notifyAll()**).
- **Timed Waiting → Runnable:** When the waiting time expires.
- **Runnable → Terminated:** When the thread completes its execution.